CS 61B — Topics, Priorities, and Workload Plan

Topics, Priorities, and Workload Plan

1. Course information

- **Course:** Computer Science 61B — *Data Structures*
- **Language:** Java
- **Course site:** https://sp26.datastructur.es/
- **Final exam: Tue May 12, 2026** — exact time/location: verify at https://sp26.datastructur.es/ before Day 1 (same day as CS 61A — the pinch point of finals week).
  - Format: in-person, written. Typically 180 min (3 hr) for the final.
  - Cheat sheet: **handwritten only** (printed/tablet-printed are confiscated). MT2 was 2 sheets; finals typically allow 3 — verify SP26.
  - Cumulative; clobber policy (final percentile can replace MT score) means the final has high leverage but is not make-or-break.
- **Core idea (one sentence):** Pick the right data structure for the job. The course is a tour of `O(log n)` and amortized `O(1)` containers (BSTs, B-trees, heaps, hash maps, tries) and the `O(V+E)` graph algorithms built on them, with a concluding section on sorting that ties asymptotic analysis to actual algorithm choice.
- **What makes the final different from MT1/MT2:** MT1 was Java + lists + inheritance. MT2 was asymptotics + disjoint sets + BSTs/B-trees/RB-trees + heaps. The **final adds graphs (5 lectures, ~biggest single block), hashing, tries, and all 5 sorting lectures** — and reuses everything from MT1/MT2 on top.

---

2. Topics — every lecture this semester

Slide deck filenames are highly descriptive. Numbering matches the SP26 schedule on sp26.datastructur.es. Two lecture numbers (30, 38) are missing in your folder — likely review/exam-day or non-content days.

MT1 territory (Lec 1–12) — Java + Lists + Inheritance

#	Lecture	Topic / main idea
01	Introduction	Course logistics, Java vs Python, "the bits don't matter, the structure does."
02	Defining/Using Classes, Lists & Arrays	Java class syntax, static vs instance, primitive types vs reference types, basic arrays.
03	Modern Lists, Arrays, Maps, References, 2D Arrays	Java collections survey, references and aliasing, 2D array memory layout.
04	Testing	JUnit basics, asserting equality, why testing matters. Conceptual; rarely heavy on exams.
05	Lists 1: Packages,	First custom list class. Recursion on linked structures. The "naked" recursive list.

#	Lecture	Topic / main idea
	Recursion, IntLists	
06	Lists 2: SLLists	Adding the helpful "wrapper" class around the bare recursive list. Sentinel nodes. Invariants.
07	Lists 3: DLLists and Basic ALists	Doubly-linked lists. First array-backed list (AList).
08	Lists 4: ArrayLists	Resizing, geometric resizing, amortized analysis preview.
09	Inheritance 1	extends, @Override, super(), polymorphism. The "is-a" relationship.
10	Inheritance 2: Iterators, Object Methods	Iterable, Iterator, equals, hashCode, toString.
11	Inheritance 3	Higher-order ideas: comparators, generic types, wildcards.
12	Midterm Practice	Practice + walkthrough — **just a checklist of MT1 coverage now.**

MT2 territory (Lec 13–20) — Asymptotics + Search Trees + Heaps

#	Lecture	Topic / main idea
13	Asymptotics I	Big-O / Big-Ω / Big-Θ definitions, common growth rates, simplification rules.
14	Data Structures 1: Disjoint Sets	Union-Find: Quick Find → Quick Union → Weighted QU → WQU + Path Compression.
15	Asymptotics II	Recurrences, amortized analysis (ALists), recursive function runtime.
16	Asymptotics III	Master Theorem, recursion trees, common pitfalls.
17	Data Structures 2: ADTs, BSTs	Abstract data types (Map, Set), binary search trees, BST invariant, Hibbard deletion.
18	Data Structures 3: B-Trees	2-3-4 trees: insert with split-and-promote, height invariant, why they're balanced.
19	Data Structures 4: Tree Rotation, Red-Black Trees	Left/right rotations, LLRBs (Left-Leaning Red-Black) as 2-3 tree mapping.
20	Data Structures 5: Priority	Heap invariant, sink/swim, heapify, removeMin / insert / peek.

#	Lecture	Topic / main idea
	Queues, Heaps	

Post-MT2 territory (Lec 21–40) — HIGHEST FINAL EMPHASIS — Graphs + Hashing + Sorting

#	Lecture	Topic / main idea
21	Graphs 1: Graphs and Traversals	Graph definitions (directed/undirected, cyclic/acyclic), adjacency representation, traversal motivation.
22	Graphs 2: DFS, BFS, Implementations	DFS recursive + iterative-with-stack, BFS with queue, when each is appropriate.
23	Graphs 3: Shortest Paths	Dijkstra's algorithm, A* search, why Dijkstra fails on negative edges.
24	Graphs 4: Minimum Spanning Trees	Prim's algorithm, Kruskal's algorithm (uses Disjoint Sets!), MST cut property.
25	Graphs 5: Directed Acyclic Graphs	Topological sort (DFS-based and Kahn's), critical path.
26	Hashing	Hash functions, hashCode contract, separate chaining, load factor.
27	Hashing II	Linear probing (open addressing), resizing, expected runtime.
28	Tries	Trie data structure for strings, R-way and TST variants, prefix lookup.
29	(lec29.pdf — likely Software Engineering intro or non-content review)	Verify on course schedule.
30	(missing)	Likely review or holiday.
(SE I)	Software Engineering	Code style, design patterns, project workflow. Conceptual.
(SE II)	Software Engineering II	More design topics. Conceptual.
32	Sorting 1: Basic Sorts	Selection sort, insertion sort, why they're $\Theta(n^2)$.
33	Sorting 2: Insertion Sort, Quicksort	Quicksort partition, recursive structure, average vs worst case.
34	Sorting 3:	In-place partition variants, quickselect (find kth element in expected

#	Lecture	Topic / main idea
	More Quicksort, Quick Select, Stability	Θ(n)), stable vs unstable.
35	Sorting 4: Theoretical Bounds on Sorting	Decision-tree argument for n log n lower bound on comparison sorts.
36	Sorting 5: Radix Sorting	Counting sort + LSD radix sort, why it can beat comparison-sort lower bound (it's not comparison-based).
37	Social Implications of Computing	Conceptual; rarely on the exam, occasionally a short essay.
38	(missing)	Possibly review.
39	Compressio n	Huffman coding, why information theory matters. Sometimes 1 short question.
40	Conclusion	Wrap; not on exam.

3. Priority list

CS 61B finals reliably have weighting roughly: graphs 25%, sorting 20%, asymptotics 15%, post-MT2 trees+heaps 15%, hashing 10%, MT1 reuse (lists/inheritance) 10%, miscellaneous 5%. Tiers below reflect that.

TIER 1 — HIGH (~55% of CS 61B time)

1. **Graph algorithms (Lec 21–25).** The single biggest block. Trace Dijkstra, Prim, Kruskal, BFS, DFS, topological sort cold. Each trace is a "follow the rules" mechanical exercise → near-full points if practiced. *Exam likelihood: ~99%.*
2. **Sorting (Lec 32–36).** Quicksort partition (3 pivot strategies on the same array), mergesort recursion tree, radix LSD trace, stability table, n log n lower bound argument. *Exam likelihood: ~99%.*
3. **Asymptotics (Lec 13, 15, 16).** Big-Θ of code snippets, recurrence solving (Master Theorem), amortized analysis. **Always tested in some form.** *Exam likelihood: ~99%.*
4. **Hashing (Lec 26–27).** Hash table operations, load factor calculation, resize trigger, linear probing vs separate chaining trace, hashCode contract. *Exam likelihood: ~85%.*
5. **BSTs / B-trees / Red-Black trees (Lec 17–19).** Insert traces, B-tree split-and-promote, rotation traces. Marcus's MT2 study guide already covers this — re-read it cold. *Exam likelihood: ~80% (still tested on final even though MT2 covered it).*

TIER 2 — MEDIUM (~30% of time)

6. **Heaps + Priority Queues (Lec 20).** Sink/swim, heapify, heap-as-array indexing. Mechanical points. *Exam likelihood: ~75%.*
7. **Disjoint Sets (Lec 14).** WQU + Path Compression. Sometimes asked standalone; always relevant for Kruskal's. *Exam likelihood: ~70%.*
8. **Tries (Lec 28).** Insert, search, prefix lookup. Smaller scope than other DS topics — high points-per-hour if you practice 2–3 problems. *Exam likelihood: ~60%.*

9. **Inheritance + Iterators + equals/hashCode (Lec 9–11).** Cumulative baseline; sometimes a "fix this code" question. *Exam likelihood: ~50%.*
10. **Lists progression (Lec 5–8).** Linked list manipulation, ALists resize logic. Foundation for amortized analysis questions. *Exam likelihood: ~55% (often folded into amortized question).*

TIER 3 — LOW (~10% of time, mostly skim)

11. **Software Engineering lectures (29, SE I, SE II).** Conceptual; usually 1 short question total or none. *Exam likelihood: ~30%.*
12. **Compression (Lec 39).** Huffman coding occasionally appears. *Exam likelihood: ~25%.*
13. **Testing (Lec 4).** Conceptual. Rare. *Exam likelihood: ~15%.*
14. **Java basics (Lec 1–3).** Cumulative baseline; you've been using Java all semester. Skim only if rusty. *Exam likelihood: ~10% standalone.*
15. **Social Implications (Lec 37).** Sometimes an essay short-answer worth a few points; if it appears, easy points. *Exam likelihood: ~30% but trivial.*

Critical leverage you already have

- Marcus's `MT2_study_guide.md` is genuinely high-quality and covers asymptotics, disjoint sets, BSTs, B-trees, RB-trees, hashing, heaps — i.e., half the final's topic surface. **Re-read it cold first; don't re-read the slides for these topics until your study guide stops triggering everything.**
- Marcus's `MT2_cheatsheet.md` — same, the cheat-sheet density version.
- 14 semesters of past finals in `Past Exams/`. Pick **Spring 2025, Fall 2024, Fall 2023, Spring 2024** as the four most representative recent finals.
- 13 discussion solutions in `Sections/`. Discs 8–13 cover post-MT2 material and are excellent practice.
- The course's own MT2 review session and worksheet PDFs.

---

4. Summary, workload anticipation, and time-allocation advice

Concise summary

CS 61B's final is **the most mechanically practice-able exam of the four**. Almost every question reduces to "execute this algorithm on this input and write down the trace." That means raw practice → raw points. Your existing MT2 study guide handles half the material; the other half (graphs + sorting + hashing) is fresh and is what your review hours should target.

Anticipated review workload (honest)

- Total hours for comfortable A-level: **14–18 hours.**
- Master plan allocates 12 hr in 5 days + 2 hr May 9 + ~2 hr May 11 evening ≈ 16 hr. **Spot-on.**
- The fact that the final is on the same day as CS 61A is the constraint — you can't extend further into May 12 morning, so the 16 hr is what you actually have.

Hour-by-hour suggested split within those ~16 hours

Activity	Hours	Why
Re-read your MT2_study_guide.md cold	0.5	Reactivate asymptotics/disjoint-sets/BSTs/B-trees/heaps.
Re-read your MT2_cheatsheet.md	0.25	Density check.
Graph algorithms drills: Dijkstra trace × 3 graphs, Prim × 2, Kruskal × 2, BFS/DFS × 2, topo sort × 2	3	Highest single block on final.
Sorting drills: quicksort partition × 3 pivots, mergesort tree, radix LSD, stability table	2	Mechanical points.

Activity	Hours	Why
Asymptotics: 10 recurrences via Master Theorem; 5 amortized-analysis problems	1.5	Always tested.
Hashing: hash table insert + linear probing + chaining traces; resize calculation	1	High yield.
Tries: insert/search/prefix on 1 example	0.5	Easy points.
Heaps: insert + removeMin × 2 inputs	0.5	Mechanical points.
Take Spring 2025 final timed (3 hr) — review every wrong answer	**3.5**	**Single highest-yield activity.**
Take Fall 2024 final timed	**3**	Confirm calibration.
Cheat sheet — handwritten 3 pages (verify allowance): condense MT2 study guide + add graph/sorting/hashing summaries	2	Writing IS review. Top of cheat sheet should have: hash table runtime table, sort comparison table, graph algorithm runtime table.

Total ≈ 16 hr.

Time-allocation advice — adjustments to the master plan

16. **Day 2 (Tue May 5) and Day 3 (Wed May 6) are your CS 61B days in the master plan.** That's 9 hr across two days for graphs + sorting + asymptotics. Use Day 2 for graphs, Day 3 for sorting + hashing. Keep MT2 review (BSTs/heaps) for short refreshers across all CS 61B sessions, not a dedicated block — your study guide handles it.
17. **Take TWO timed past finals, not one.** The master plan implies one (Day 4 Thu evening or May 9 Sat). With 14 semesters of past exams available, a second timed attempt on May 9 will calibrate you better. Replace the May 11 evening "last drill" (which is exhausted-brain time) with a quiet light review of mistakes from your two timed exams.
18. **Cheat sheet writing happens BETWEEN the two timed exams**, not before either. Take exam 1 → identify gaps → write cheat sheet targeting those gaps → take exam 2 → final cheat-sheet edits → exam day.
19. **The May 11 evening "CS 61A + CS 61B last drill" is dangerous** — you'll have just finished ECON 100B and your cognitive bandwidth will be low. Limit it to ~1 hour of cheat-sheet review and graph-algorithm trace warm-ups, not new practice. Sleep is a study tool here.

What NOT to do (anti-advice)

- Don't re-read all 39 lecture slides. Your MT2 study guide replaces ~half. The remaining half is in slide deck titles 21–28 and 32–36.
- Don't try to memorize the LLRB rotation rules from scratch — they're a *consequence* of B-tree mapping. Understand the mapping (LLRB ↔ 2-3 tree) and the rotations follow.
- Don't print your cheat sheet from a tablet. The course confiscates these (per your `MT2_study_guide.md`).
- Don't do all 14 past finals. Two recent ones, timed, with thorough review beats four rushed ones.
- Don't cram new material on the morning of May 12. By then everything either lives in your cheat sheet or it doesn't.